GENERATIVE AI

EXPT 1A. Text Generation using N-gram Language model

Code:

```
# ----------- INSTALL & IMPORT -----------
import nltk
import random
from collections import defaultdict
from nltk.util import ngrams
from collections import Counter

# Download dataset directly from NLTK servers (NO MANUAL DOWNLOAD)
nltk.download('brown')
nltk.download('universal_tagset')
nltk.download('punkt') # General punkt tokenizer
nltk.download('punkt_tab') # Specifically for the punkt_tab resource
mentioned in the error

from nltk.corpus import brown
from nltk.tag import hmm


# ----------- LOAD DATA -----------
print("Loading Brown Corpus...")
words = brown.words()
sentences = brown.sents()
tagged_sentences = brown.tagged_sents(tagset='universal')

print("Total words:", len(words))
print("Total sentences:", len(sentences))


# ----------- PREPROCESSING -----------
tokens = [word.lower() for word in words if word.isalpha()]

print("Total processed tokens:", len(tokens))
```

```
[nltk_data] Downloading package brown to /root/nltk_data...
[nltk_data]   Package brown is already up-to-date!
[nltk_data] Downloading package universal_tagset to /root/nltk_data...
[nltk_data]   Package universal_tagset is already up-to-date!
[nltk_data] Downloading package punkt to /root/nltk_data...
[nltk_data]   Unzipping tokenizers/punkt.zip.
[nltk_data] Downloading package punkt_tab to /root/nltk_data...
[nltk_data]   Unzipping tokenizers/punkt_tab.zip.
Loading Brown Corpus...
Total words: 1161192
Total sentences: 57340
Total processed tokens: 981716
```

```python
# ----------- N-GRAM MODEL -----------
# Function to build N-gram model
def build_ngram_model(tokens, n):
    model = defaultdict(list)
    for gram in ngrams(tokens, n):
        context = gram[:-1]
        prediction = gram[-1]
        model[context].append(prediction)
    return model

# Function to predict the next word using an N-gram model
def predict_next_ngram(text, model):
    context_words = tuple(text.lower().split())

    # Adjust context length to match the model's n-gram order
    # For bigram, context_words should be 1 word. For trigram, 2 words.
    n_minus_1 = len(list(model.keys())[0]) if model else 0 # Infer n-1
from model keys

    if n_minus_1 > 0: # Ensure model is not empty and context is
meaningful
        context = context_words[-n_minus_1:]
    else:
        context = context_words # Fallback for unigram or if model is
empty

    if context in model:
        return random.choice(model[context])
    else:
        # Fallback to a shorter n-gram if available, or just a random word
```

```
        if n_minus_1 > 1: # If it was a trigram or higher, try a
bigram-like context
            return predict_next_ngram(text.split()[-1],
build_ngram_model(tokens, 2)) if len(text.split()) > 1 else "No prediction
available"
        return "No prediction available"

print("\nBuilding N-Gram Models...")
bigram_model = build_ngram_model(tokens, 2) # context: 1 word, predicts 1
word
trigram_model = build_ngram_model(tokens, 3) # context: 2 words, predicts
1 word

print("\nN-GRAM MODEL TESTING:")
context_1_bigram = "the"
context_2_trigram = "in the"

print(f"Bigram ('{context_1_bigram}') next word prediction:
{predict_next_ngram(context_1_bigram, bigram_model)}")
print(f"Trigram ('{context_2_trigram}') next word prediction:
{predict_next_ngram(context_2_trigram, trigram_model)}")
```

```
Building N-Gram Models...

N-GRAM MODEL TESTING:
Bigram ('the') next word prediction: highest
Trigram ('in the') next word prediction: world
```

```
# ----------- N-GRAM INTERACTIVE DEMO -----------
print("\n=========== N-GRAM INTERACTIVE MODE ==========")
print("Type a word or two (e.g., 'the', 'in the') for N-gram prediction
(type 'exit' to stop):")

while True:
    user_input = input("\nEnter text: ")
    if user_input.lower() == "exit":
        break

    words_count = len(user_input.split())
```

```
    if words_count == 1:
        prediction = predict_next_ngram(user_input, bigram_model)
        print(f"N-Gram (Bigram) Prediction for '{user_input}':
{prediction}")
    elif words_count >= 2:
        prediction = predict_next_ngram(user_input, trigram_model)
        print(f"N-Gram (Trigram) Prediction for '{user_input}':
{prediction}")
    else:
        print("Please enter at least one word for prediction.")
```

```
=========== N-GRAM INTERACTIVE MODE ==========
Type a word or two (e.g., 'the', 'in the') for N-gram prediction (type 'exit' to stop):

Enter text: the
N-Gram (Bigram) Prediction for 'the': death

Enter text: in
N-Gram (Bigram) Prediction for 'in': day

Enter text: exit
```

EXPT 1B. Hidden Markov Model (HMM) based Predictive Text System

Code:

```
# ----------- HMM MODEL -----------
# Prepare data for HMM training
# Split tagged_sentences into training and testing (optional, but good
practice)
train_size = int(len(tagged_sentences) * 0.9)
training_data = tagged_sentences[:train_size]
testing_data = tagged_sentences[train_size:]

print(f"\nHMM Training with {len(training_data)} sentences and Testing
with {len(testing_data)} sentences.")

# Train the HMM Tagger
# This might take a few moments
print("Training HMM Tagger... (This may take a while)")
hmm_tagger = hmm.HiddenMarkovModelTagger.train(training_data)
print("HMM Tagger training complete.")
```

```python
# Evaluate the HMM Tagger (optional)
# accuracy = hmm_tagger.evaluate(testing_data)
# print(f"HMM Tagger Accuracy: {accuracy:.2f}")

# Build POS -> word mapping from training data
pos_word_map = defaultdict(list)
for sent in training_data:
    for word, tag in sent:
        pos_word_map[tag].append(word)

# Function to predict next words based on HMM tagged POS
def predict_next_hmm(sentence):
    tokens = nltk.word_tokenize(sentence.lower()) # Tokenize input
sentence
    try:
        # Tag the input sentence to get the POS tag of the last word
        tagged_sentence = hmm_tagger.tag(tokens)
        last_word_tag = tagged_sentence[-1][1] # Get tag of the last word

        # Get words commonly associated with that POS tag
        if last_word_tag in pos_word_map:
            word_freq = Counter(pos_word_map[last_word_tag])
            # Return top 5 most common words for that tag
            return [w for w, _ in word_freq.most_common(5)]
        else:
            return ["No prediction (tag not found)"]
    except Exception as e:
        # Handle cases where tagging might fail (e.g., unknown words)
        # print(f"HMM prediction error: {e}") # For debugging
        return ["No prediction available (HMM error)"]

print("\nHMM MODEL TESTING:")
context_hmm = "The quick brown fox"
print(f"HMM ('{context_hmm}') next word predictions:
{predict_next_hmm(context_hmm)}")
context_hmm_2 = "She is running"
print(f"HMM ('{context_hmm_2}') next word predictions:
{predict_next_hmm(context_hmm_2)}")
```

```
HMM Training with 51606 sentences and Testing with 5734 sentences.
Training HMM Tagger... (This may take a while)
HMM Tagger training complete.

HMM MODEL TESTING:
HMM ('The quick brown fox') next word predictions: ['time', 'man', 'Af', 'years', 'Mr.']
HMM ('She is running') next word predictions: ['is', 'was', 'be', 'had', 'are']
```

```python
# ----------- HMM INTERACTIVE DEMO -----------
print("\n=========== HMM INTERACTIVE MODE ==========")
print("Type a sentence for HMM-based next word prediction (type 'exit' to stop):")

while True:
    user_input = input("\nEnter sentence: ")
    if user_input.lower() == "exit":
        break

    if user_input.strip() == "":
        print("Please enter a sentence.")
        continue

    predictions = predict_next_hmm(user_input)
    print(f"HMM Prediction for '{user_input}': {', '.join(predictions)}")
```

```
=========== HMM INTERACTIVE MODE ==========
Type a sentence for HMM-based next word prediction (type 'exit' to stop):

Enter sentence: she is running
HMM Prediction for 'she is running': is, was, be, had, are

Enter sentence: the quick brown fox
HMM Prediction for 'the quick brown fox': time, man, Af, years, Mr.

Enter sentence: exit
```

GOOGLE COLLAB LINK : GenAI EX-1.ipynb